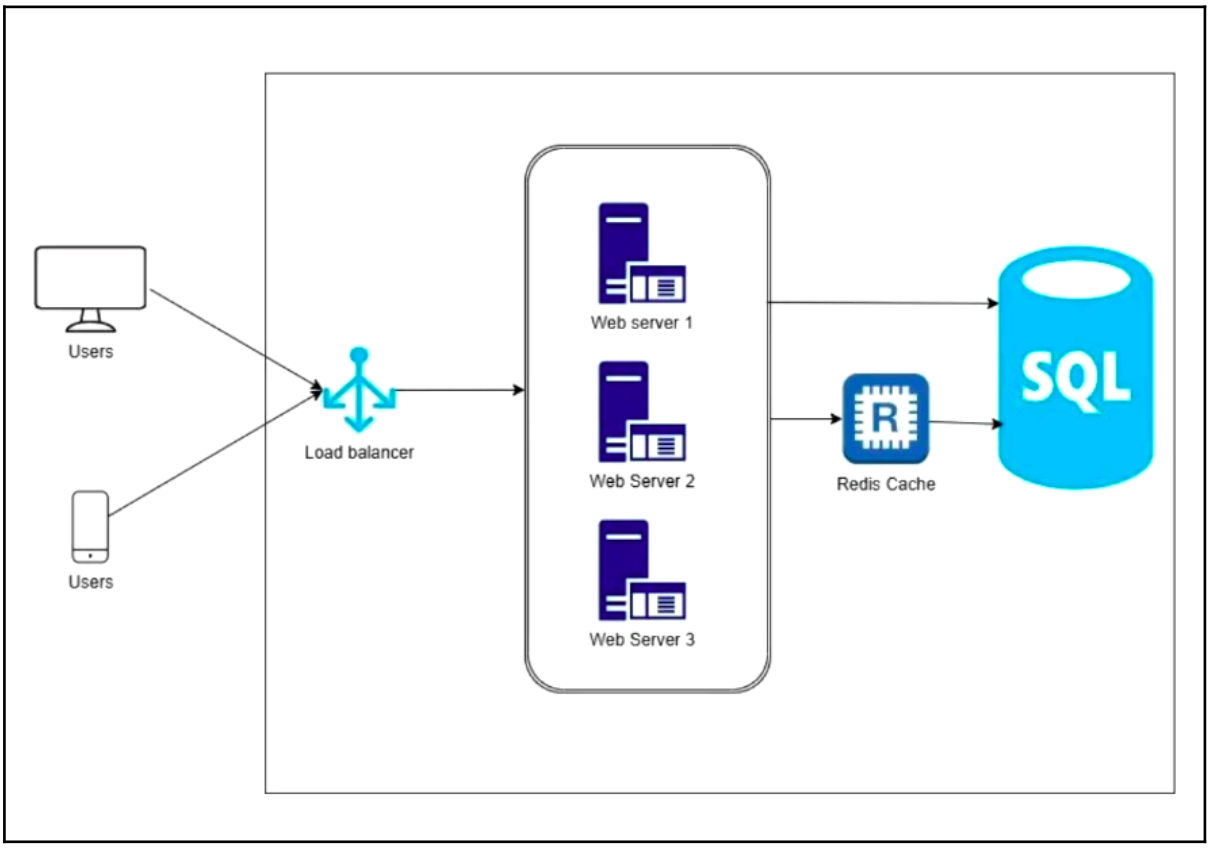
Section 1: Introduction

What is System Design?

System Design is the process of defining the architecture, components, modules, interfaces, and data flow for a system to meet specific requirements. Basically, at its core system design is the process of defining how a system should be structured, which includes its overall architecture, what all the components should be there, how many modules should be there in the system, how the interfaces should look like, how the data should flow between the module and components to meet any specific requirement.



System design is not just about drawing the diagrams on a whiteboard or on a document. It is more about critical thinking and designing the high level architecture, taking those high level decisions that impact not just the functional, but also the non-functional requirements of the application.These are requirements like scalability, reliability, performance and maintainability of a system.

Most of the applications we use in our daily lives are scalable, distributed and have huge databases. They are running on some cloud infrastructure and behind each of these applications, there is a well thought out system design that somebody must have done before creating the application, and probably somebody is still thinking about design to tackle new scalability, performance, and caching-related issues.

A good system design ensures that your system runs efficiently, it handles massive loads, and it automatically recovers from failures smoothly.

Why is System Design Important?

A lot of people approach system design with just one goal which is to crack interviews. But system design is so much more than that. It's actually a very critical skill that is required for building efficient software systems. Understanding the system design principles will naturally help you crack interviews. This will help you to architect real world solutions effectively.

Some of the key reasons why the system design actually matters are:

1. **Scalability & Reliability:** It helps to build scalable and reliable systems, ensuring that it can handle millions of users without failure. A good system design is crucial where it handles millions and billions of users. Without a good design, the system will crash under heavy load. A well designed system ensures high availability, fault tolerance and performance optimizations. Learning about system design gives us the ammunition required to create such scalable applications.

2. **Architectural Thinking:** Learning system design goes beyond coding, as you are gonna be thinking at an architectural level. Writing efficient code is important but real world engineering problems go beyond just writing code. System design helps engineers think at an architectural level. It allows us to make trade-offs like choosing SQL or NoSQL, microservice or monolith, or balancing between consistency or availability and understanding these concepts will make us a better engineer.

3. **Career Growth:** Career growth is self-evident as most people actually start learning system design not just from an interview perspective, but also to become a senior level engineer or architect to focus on system design. Junior engineers mostly focus on writing codes, while senior engineers and architects focus on scalability, maintainability and the overall system health. Mastering system design is very essential if you are aspiring to be a tech leader architect. Most companies expect senior engineers to think beyond individual features and should be able to design robust and scalable systems.

4. **Real World Problem Solving:** Most people make mistakes by just memorizing the system design concept and mug up for the interviews. In reality or in real world projects, that knowledge is likely to fall apart. We need to understand the system design fundamentally, conceptually to be very strong at it. System design is about solving real world engineering challenges and not just passing interviews. When you truly understand what system design is, why systems are designed in a certain way, then only you will be able to design good systems and also cracking interviews becomes easy.

5. **Trade-Offs & Decision Making:** System design is predominantly about making informed trade-off decisions. There is no such thing as a perfect system design. Only well reasoned trade offs are there in a good system design. Engineers should learn to balance scalability, cost, speed and complexity to achieve their functional and non-functional requirements at that point of time. For example, choosing a SQL or NoSQL, it isn't about a personal preference. It depends on the functional requirements of the application. Informed trade offs is one of the things that learning system design will help us in.

6. **Future-Proofing:** We know that technology is constantly evolving. Poorly designed systems become bottlenecks over time. A well designed system, on the other hand, can adapt to growing traffic, new features and emerging technologies. Companies like netflix, and ebay started with monolithic architecture, but later transitioned to microservices as they scaled. By learning system design as a skill rather than just a checklist for the interviews, you will be prepared for both technical interviews and real-world engineering challenges. So instead of just studying system design from an interview-centric perspective, it's better to truly master the system design and build scalable, efficient and future-proof systems.